

Apache Hive Lab

Slowly Changing Dimension Type 2

Implementing historical tracking without UPDATE/DELETE

Eng. Mohammed Al-Zubiri

Objective: Build a customer dimension table that keeps full history (SCD Type 2) – no transactional tables, no ACID, no standard updates.

1 Creating Tables and Solving the Delimiter Issue

1.1 The Problem with Commas in Addresses

When I first tried to load the CSV files with a simple comma separator, Hive would rip apart fields that contained commas – like addresses in the address column. A row like “36923 Bowers Gateway Suite 027 New Kristi, MP 44312” would be split across two columns, completely misaligning the schema.

1.2 Solution: OpenCSVSerde

To handle the commas properly I switched to the OpenCSVSerde. This serializer/deserializer understands quoted fields and keeps the commas that appear inside double quotes, which is exactly what we need for this data. I didn’t have to change the separator in the source files, only the table definition.

1.3 Internal Table – customer_internal

I created an internal (managed) table with the OpenCSVSerde, then loaded the customer_updated.csv file using the local file system.

```

1 CREATE TABLE customer_internal (
2   CustomerID INT,
3   Name STRING,
4   Email STRING,
5   Phone_Number STRING,
6   Address STRING,
7   JOIN_Date STRING
8 )
9 ROW FORMAT SERDE 'org.apache.hadoop.hive.serde2.OpenCSVSerde'
10 WITH SERDEPROPERTIES (
11   "separatorChar" = ",",
12   "quoteChar" = "\"",
13   "escapeChar" = "\\"
14 )
15 STORED AS TEXTFILE
16 TBLPROPERTIES ("skip.header.line.count"="1");
17
18 LOAD DATA LOCAL INPATH '/home/itversity/csv/customer_updated.csv' INTO TABLE customer_internal;
19
20 SELECT * FROM customer_internal; 499ms

```

customerid	name	email	phone_number	address	join_date
12346	Debra Wells	michaelkerr@example.net	3317035462	USNV Olsen FPO AA 43254	2025-02-10
12347	Michael Grimes	rhandler@example.net	243.521.3090	36923 Bowers Gateway Suit	2025-03-14
12348	Jennifer Cuevas	brandonbrown@example.cr	+1-863-584-5200x273	095 Stevenson Lodge New I	2025-01-03

Figure 1: CREATE TABLE and LOAD DATA for internal table.

1.4 External Table – customer_external

The external table was defined with the same SerDe, but pointed to a fixed HDFS location. After creating it, I manually copied the CSV into that directory with `hdfs dfs -put`.

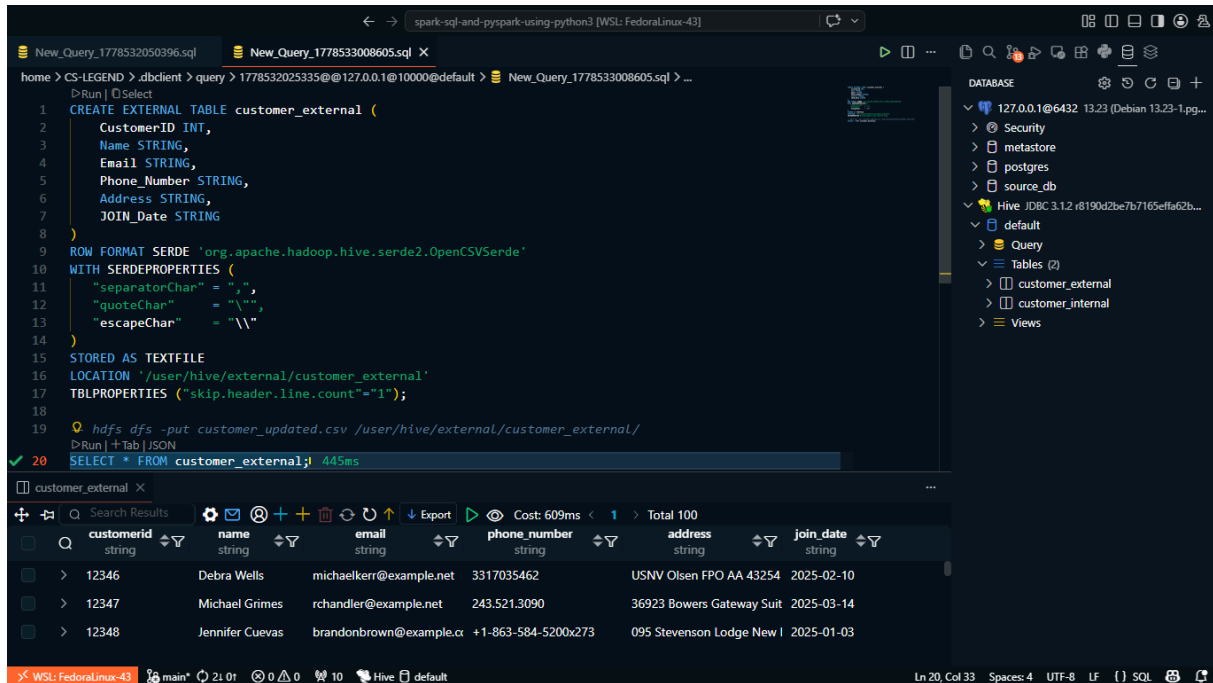


Figure 2: External table definition and HDFS file placement.

```

itversity@itvdelab:~/csv$ hdfs dfs -put customer_updated.csv /user/hive/external/customer_external/
itversity@itvdelab:~/csv$ hdfs dfs -ls /user/hive/external/customer_external/
Found 1 items
-rw-r--r-- 1 itversity supergroup 513165 2026-05-11 20:58 /user/hive/external/customer_external/customer_updated.csv
itversity@itvdelab:~/csv$

```

Figure 3: HDFS put command and directory listing.

2 Dropping Tables: Internal vs. External Behaviour

I dropped both tables to see how Hive treats the underlying data.

2.1 Internal Table (customer_internal)

Dropping an internal table removes both the metadata and the data files. After `DROP TABLE customer_internal;`, the folder under `/user/hive/warehouse` disappeared – the data was gone for good.

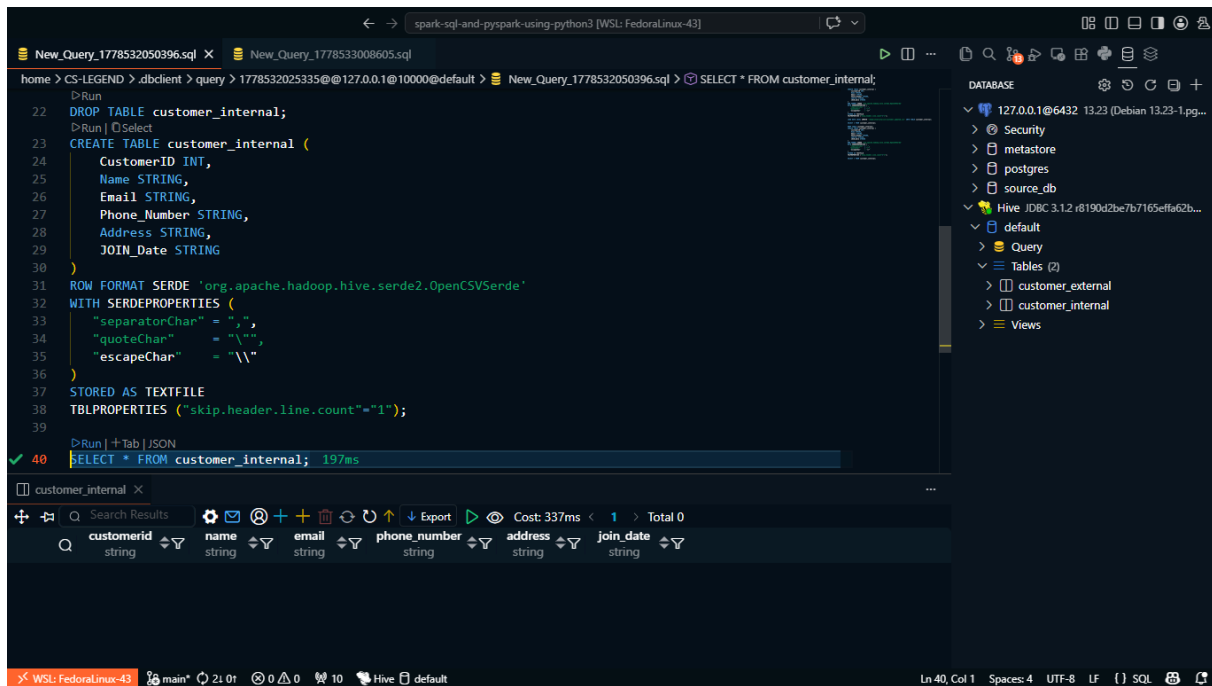


Figure 4: After dropping the internal table: data erased from HDFS.

2.2 External Table (customer_external)

Dropping an external table only deletes the metadata from the Hive metastore. The data files in `/user/hive/external/customer_external` remained untouched. I could re-create the table with the same location and query the data again immediately.

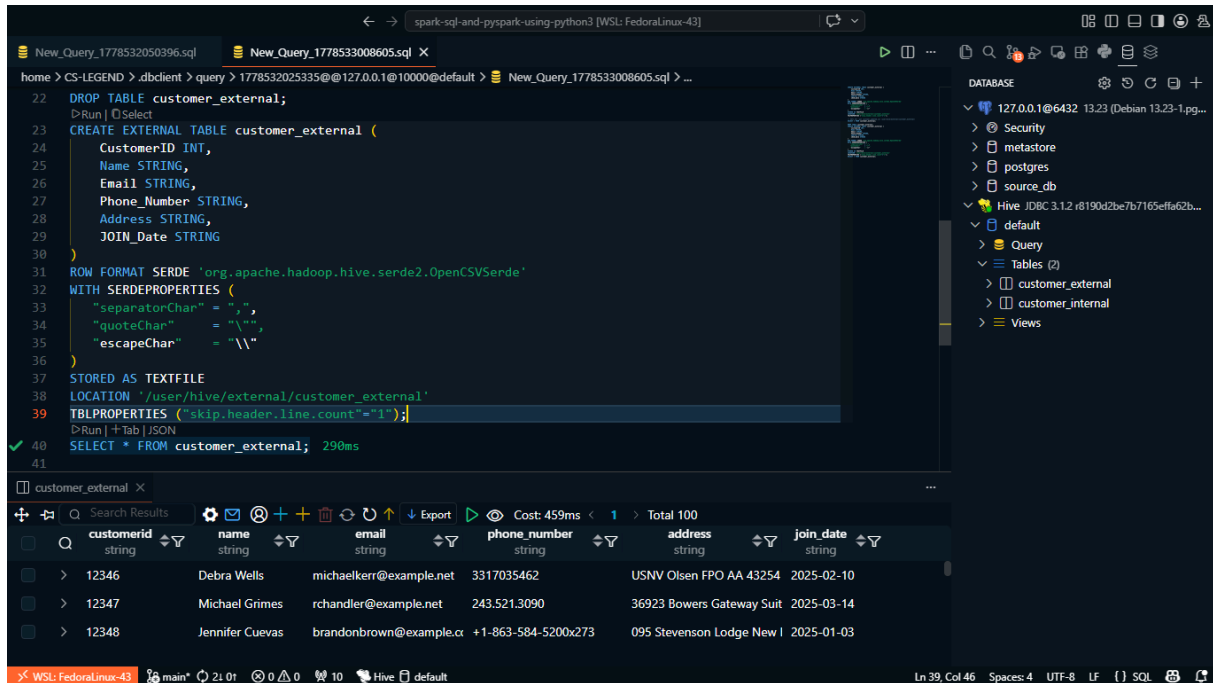


Figure 5: External table dropped, but files still inside the HDFS directory.

This confirms why external tables are safer when multiple processes share raw data.

3 Slowly Changing Dimension (SCD) Type 2 Workaround

3.1 The Core Challenge

Hive's non-transactional tables do not allow UPDATE or DELETE. For a SCD Type 2 I need to expire old rows and insert new ones – operations that normally rely on UPDATE. The workaround: rebuild the entire table in one shot using INSERT OVERWRITE combined with UNION ALL. I compute the desired final state of the dimension and replace the old table with that result.

3.2 Step 1: Initial Dimension Load

I created `customer_dim` as an internal table and loaded it with the base data from `customer_scd2_mixed.csv`. This file already contained some inactive records (with `Is_Current = 0`) and active ones (`Is_Current = 1`).

The screenshot shows a SQL IDE with the following SQL code:

```

1 CREATE TABLE customer_dim (
2   CustomerID INT,
3   Name STRING,
4   Email STRING,
5   Phone_Number STRING,
6   Address STRING,
7   JOIN_Date STRING,
8   Start_Date STRING,
9   End_Date STRING,
10  Is_Current INT
11 )
12 ROW FORMAT SERDE 'org.apache.hadoop.hive.serde2.OpenCSVSerde'
13 WITH SERDEPROPERTIES ("separatorChar"=";", "quoteChar"="\")
14 STORED AS TEXTFILE
15 TBLPROPERTIES ("skip.header.line.count"="1");
16
17 LOAD DATA LOCAL INPATH '/home/itversity/csv/customer_scd2_mixed.csv' INTO TABLE customer_dim;
18
19 SELECT * FROM customer_dim; 451ms

```

The right sidebar shows the database structure:

- 127.0.0.1@6432 13.23 (Debian 13.23-1.pg...
- Security
- metastore
- postgres
- source_db
- Hive JDBC 3.1.2 r8190d2be7b7165effa62b...
- default
 - Query
 - Tables (3)
 - customer_dim
 - customer_external
 - customer_internal
 - Views

The bottom section shows the results of the SELECT query:

customerid	name	email	phone_number	address	join_date	start_date
16324	Jennifer Harris	upd_michael77@example.n	001-438-988-5271x923	323 Yoder Place East Kather	2/3/2025	4/1/202
13094	Lisa Morris	upd_brandtscott@example.	459.507.0988	85587 Cook Flats Jamesfurt	3/6/2025	4/1/202
13483	Jesus Cochran	upd_natalie29@example.co	001-911-846-8022	57000 Nancy Crescent Apt.	1/18/2025	4/1/202

Figure 6: Initial dimension table creation and data load.

3.3 Step 2: Staging Table for Updates

I created a separate `customer_updates` table and loaded the `customer_updates.csv` file. This table holds the latest changes – new customers and modifications to existing ones.

The screenshot displays a SQL IDE interface with the following components:

- Query Editor:** Contains SQL code to create the `customer_updates` table and load data from a local CSV file.


```

1 CREATE TABLE customer_updates (
2   CustomerID INT,
3   Name STRING,
4   Email STRING,
5   Phone_Number STRING,
6   Address STRING,
7   JOIN_Date STRING
8 )
9 ROW FORMAT SERDE 'org.apache.hadoop.hive.serde2.OpenCSVSerde'
10 WITH SERDEPROPERTIES ("separatorChar"=",", "quoteChar"="\")
11 STORED AS TEXTFILE
12 TBLPROPERTIES ("skip.header.line.count"="1");
13
14 LOAD DATA LOCAL INPATH '/home/itversity/csv/customer_updated.csv' INTO TABLE customer_updates;
15
16 SELECT * FROM customer_updates;
      
```
- Database Explorer:** Shows the database structure, including the `customer_updates` table under the `default` schema.
- Table View:** Displays the data loaded into the `customer_updates` table.

customerid	name	email	phone_number	address	join_date
12346	Debra Wells	michaelkerr@example.net	3317035462	USNV Olsen FPO AA 43254	2025-02-10
12347	Michael Grimes	rhandler@example.net	243.521.3090	36923 Bowers Gateway Suit	2025-03-14
12348	Jennifer Cuevas	brandonbrown@example.co	+1-863-584-5200x273	095 Stevenson Lodge New I	2025-01-03

Figure 7: Staging table containing incoming changes.

3.4 Step 3: The INSERT OVERWRITE Query

The final query is a single `INSERT OVERWRITE TABLE customer_dim` that pulls together three groups of data via `UNION ALL`:

1. **Untouched records** – rows from the existing dimension that are either already historically inactive (`Is_Current = '0'`) or have no matching update at all. A `LEFT JOIN` with `customer_updates` isolates those that should stay exactly as they are.
2. **Expired old versions** – for every customer that appeared in the update staging table **and** was currently active in the dimension (`Is_Current = '1'`), I create a new row with `End_Date` set to today, marking it as historical.
3. **New and updated records** – from the staging table I generate brand-new active rows with `Start_Date` today, `End_Date` null, and `Is_Current = 1`.

```

21 -- SCD2 Overwrite Query
22 INSERT OVERWRITE TABLE customer_dim
23
24 -- GROUP 1: Untouched Records (No updates came in, or already historically inactive)
25 D:Run | + Tab | JSON | O Select
26 SELECT
27     d.CustomerID, d.Name, d.Email, d.Phone_Number, d.Address, d.JOIN_Date, d.Start_Date, d.End_Date, d.Is_Current
28 FROM customer_dim d
29 LEFT JOIN customer_updates u
30     ON d.CustomerID = u.CustomerID
31 WHERE u.CustomerID IS NULL
32     OR d.Is_Current = '0'
33
34 UNION ALL
35
36 -- GROUP 2: Expired Records (An update arrived, so we close out the old active record)
37 D:Run | + Tab | JSON | O Select
38 SELECT
39     d.CustomerID, d.Name, d.Email, d.Phone_Number, d.Address, d.JOIN_Date, d.Start_Date, CAST(current_date()) AS
40     STRING AS End_Date, '0' AS Is_Current
41 FROM customer_dim d
42 JOIN customer_updates u
43     ON d.CustomerID = u.CustomerID
44 WHERE d.Is_Current = '1'
45
46 UNION ALL
47
48 -- GROUP 3: New and Updated Records (Coming from the update file, marked as active)
49 D:Run | + Tab | JSON
50 SELECT
51     u.CustomerID, u.Name, u.Email, u.Phone_Number, u.Address, u.JOIN_Date, CAST(current_date()) AS STRING AS
52     Start_Date, CAST(NULL AS STRING) AS End_Date, '1' AS Is_Current
53 FROM customer_updates u; AffectedRows: -1 1m 17.3s

```

Figure 8: The complete SCD2 overwrite query using `UNION ALL` of three logical groups.